



Memoria Tecnica LoveCode

Desarrollo de aplicaciones web

Programación, Bases de Datos, Sistemas informáticos,

Lenguaje de Marcas, Entornos de desarrollo

Jose Luis Lopez De Pablo Moya, Esteban Castañer, Pedro Campos

Pablo Greus



Índice

1. Introducción y objetivos del Proyecto.....	4
2. Descripción general del Sistema	4
2.1. Capa de Presentación Frontend	5
2.2. Capa de Lógica Backend	5
2.3. Capa de Datos Base de Datos.....	5
2.4. Comunicación entre Capas.....	5
3. Diseño de la Base de Datos.....	6
3.1. Tabla Usuario	6
3.2. Tabla Tecnología	6
3.3. Tabla Usuarios_Tecnologías	6
3.4. Tabla Likes	7
3.5. Tabla Matches	8
3.6. Trigger: after_like_insert	8
3.7. Procedimientos Almacenados.....	9
3.7.1. Insertar_usuarios	9
3.7.2. BorrarUsuario.....	9
3.7.3. CargarUsuario	10
3.8. Esquema Entidad Relación (EER).....	11
4. Desarrollo del Backend (Java/Spring Boot).....	12
4.1. Conexión JDBC	12
4.2. Registro de Usuario	13
4.3. Validación del Login.....	13
4.4. Obtención de Perfiles para el Dashboard	13
4.5. Registro del Likes y Detección de Match.....	14
4.6. Spring Boot.....	15
4.6.1 Pom.xml	15
4.6.2 Punto de entrada/ Main.java	16
4.6.3 Configuración CORS	17
5. Desarrollo del Frontend.....	18
5.1. Sistema de Diseño Visual.....	19
5.2. Peticiones HTTP Asincronas con fetch()	19



5.3. Motor de Swipe	20
6. Backup	20
6.1. Backup desde entorno grafico	20
6.2. Backup manual desde la powershell.....	22
6.3. Backup Automatizado.....	22
7. Docker.....	23
8. Funcionamiento de la web	24
Paso 1 Acceso a la aplicacion	24
Paso 2 Registro	24
Paso 4 Exploración de perfiles.....	24
Paso 3 Inicio de Sesion	25
Paso 5 Like y detención de Match	25
9. Problemas Y soluciones.....	26
Fallo en maquina virtual	26
Fallo en tabla usuario.....	26
Error 500 en el endpoint de matches.	26
Los Likes no se insertaban en la base de datos	26



1. Introducción y objetivos del Proyecto

LoveCode es una aplicación web de tipo matching inspirada en la mecánica de deslizamiento de tarjetas popularizada por aplicaciones de citas. LoveCode adapta este concepto al ecosistema tecnológico con un objetivo claro: facilitar que los desarrolladores encuentren otros perfiles afines, ya sea para colaborar en proyectos, formar equipos de trabajo, buscar mentores o simplemente conectar con programadores que comparten sus mismas tecnologías y pasiones.

El problema que resuelve LoveCode es real y frecuente en la comunidad tecnológica: los desarrolladores, especialmente los más jóvenes o los que están iniciándose en el sector, tienen dificultades para encontrar compañeros de proyectos habilidades complementarias o similares. Las plataformas profesionales existentes, como LinkedIn o GitHub, están orientadas al ámbito formal y no ofrecen un mecanismo ágil e intuitivo para este tipo de conexión informal pero productiva.

LoveCode resuelve este problema mediante un flujo sencillo y atractivo: el usuario se registra indicando su nombre, correo, contraseña, una breve descripción personal (bio) y las tecnologías que domina o con las que trabaja habitualmente. Una vez dentro de la aplicación, puede explorar los perfiles de otros desarrolladores presentados en forma de tarjetas interactivas. Si un perfil le resulta interesante, desliza la tarjeta hacia la derecha o pulsa el botón de like. Si la otra persona también le ha dado like, el sistema detecta la coincidencia mutua y genera automáticamente un match, notificando a ambos usuarios.

Las funcionalidades principales de la aplicación son las siguientes:

- Registro de usuarios con selección de tecnologías mediante una interfaz de chips interactivos (JavaScript, Python, Java, TypeScript, SQL, React, CSS, C#).
- Autenticación mediante correo electrónico y contraseña, con persistencia de sesión en el navegador.
- Dashboard de perfiles con motor de swipe: arrastre con ratón, gestos táctiles y atajos de teclado.
- Sistema de likes con detección automática de match mutuo en el backend.
- Página de matches que muestra los perfiles con los que se ha producido una coincidencia, incluyendo bio, tecnologías y fecha del match.
- Interfaz responsiva con identidad visual coherente basada en un sistema de diseño propio.

2. Descripción general del Sistema

El sistema se organiza siguiendo una arquitectura cliente-servidor en cuatro capas bien diferenciadas.



2.1. Capa de Presentación | Frontend

La capa de presentación está formada por cinco páginas HTML estáticas servidas directamente en el navegador del usuario, junto con sus correspondientes hojas de estilo CSS y módulos JavaScript. No requiere ningún framework de frontend: toda la lógica de interacción, las animaciones de swipe y las peticiones al servidor están implementadas en JavaScript vanilla.

2.2. Capa de Lógica | Backend

El servidor está desarrollado en Java utilizando el framework Spring Boot, que simplifica la creación de APIs REST eliminando gran parte de la configuración manual. El backend expone once endpoints HTTP bajo el prefijo /api, que el frontend consume mediante la función nativa fetch() del navegador. Cada petición y respuesta se serializa en formato JSON. El backend es responsable de la validación de datos de entrada, la gestión de la lógica (detección de matches mutuos, normalización de tecnologías) y la comunicación con la base de datos mediante JDBC.

2.3. Capa de Datos | Base de Datos

La persistencia de datos se gestiona con un sistema gestor de bases de datos relacional compatible con MySQL. La base de datos, denominada LoveCode, contiene cinco tablas relacionadas entre sí mediante claves foráneas con integridad referencial. El servidor de base de datos se levanta mediante Docker, lo que garantiza un entorno reproducible e independiente del sistema operativo del desarrollador.

2.4. Comunicación entre Capas

La comunicación entre el frontend y el backend se realiza exclusivamente mediante peticiones HTTP con cuerpo en formato JSON. . El navegador envía peticiones POST para operaciones de escritura (registro, login, like) y peticiones GET para operaciones de lectura (listado de perfiles, listado de matches). El backend responde con objetos JSON que el JavaScript del frontend parsea y usa para actualizar la interfaz de usuario dinámicamente, sin necesidad de recargar la página.

La comunicación entre el backend y la base de datos se realiza mediante JDBC (Java Database Connectivity), la API estándar de Java para la conexión y ejecución de consultas SQL en bases de datos relacionales. Cada clase DAO gestiona sus propias conexiones usando DriverManager.getConnection() con las credenciales definidas en la clase Variables.java

Capa	Tecnología	Comunicación con la siguiente capa
Frontend	HTML,CSS,JavaScript	HTTP,JSON mediante fetch()



Backend	Java , Spring Boot	JDBC/ SQL mediante DriverManager
Base de Datos	SQL	

Tabla 1. Resumen de capas.

3. Diseño de la Base de Datos

La base de datos esta diseñada en 5 tablas:

3.1. Tabla Usuario

Es la entidad central del sistema. Almacena los datos de cada desarrollador registrado en la plataforma. Su clave primaria es `id_usuario`, un entero con autoincremento. El campo correo tiene una restricción `UNIQUE` que impide el registro de dos cuentas con el mismo correo electrónico. El campo `fecha_registro` se rellena automáticamente con la fecha y hora actuales mediante `DEFAULT NOW()`.

```
CREATE TABLE usuario (  
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(255) NOT NULL,  
  correo VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  bio text,  
  fecha_registro datetime not null default now()  
);
```

3.2. Tabla Tecnología

Actúa como catálogo normalizado de tecnologías disponibles en la plataforma. Al separar las tecnologías en una tabla propia en lugar de almacenarlas como texto libre en el perfil del usuario, se evita la redundancia y se facilita la búsqueda y el filtrado. El campo nombre tiene restricción `UNIQUE` para evitar duplicados en el catálogo.

```
CREATE TABLE Tecnologia (  
  id_tecnologia INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(50) NOT NULL UNIQUE  
);
```

3.3. Tabla Usuarios_Tecnologías

Es la tabla de relación N:M entre usuario y Tecnología. Permite que un usuario tenga múltiples tecnologías y que una misma tecnología pertenezca a múltiples usuarios. Su clave primaria es compuesta (`id_usuario`, `id_tecnologia`), lo que garantiza que no puede



existir el mismo vínculo duplicado. Ambas columnas son claves foráneas con ON DELETE CASCADE, de modo que si se elimina un usuario o una tecnología, los vínculos correspondientes se eliminan automáticamente.

```
CREATE TABLE Usuarios_Tecnologias (  
    id_usuario INT NOT NULL,  
    id_tecnologia INT NOT NULL,  
    PRIMARY KEY (id_usuario, id_tecnologia),  
  
    CONSTRAINT fk_usuario FOREIGN KEY (id_usuario)  
    REFERENCES usuario(id_usuario)  
    ON DELETE CASCADE,  
  
    CONSTRAINT fk_tecnologia FOREIGN KEY (id_tecnologia)  
    REFERENCES Tecnologia(id_tecnologia)  
    ON DELETE CASCADE  
);
```

3.4. Tabla Likes

Registra cada acción de like emitida por un usuario hacia otro. Las columnas id_emisor e id_receptor son claves foráneas que referencian a la tabla usuario. La restricción UNIQUE(id_emisor, id_receptor) impide que el mismo usuario pueda dar like dos veces al mismo perfil. El campo fecha registra el momento exacto de la acción.

```
CREATE TABLE Likes (  
    id_like INT NOT NULL AUTO_INCREMENT,  
    id_emisor INT NOT NULL,  
    id_receptor INT NOT NULL,  
    fecha DATETIME NOT NULL DEFAULT NOW(),  
  
    PRIMARY KEY (id_like),  
    UNIQUE (id_emisor, id_receptor),  
  
    CONSTRAINT fk_emisor FOREIGN KEY (id_emisor)  
    REFERENCES usuario(id_usuario) ON DELETE CASCADE,  
  
    CONSTRAINT fk_receptor FOREIGN KEY (id_receptor)  
    REFERENCES usuario(id_usuario) ON DELETE CASCADE  
);
```



3.5. Tabla Matches

Almacena los pares de usuarios que se han dado like mutuamente. Para garantizar que un mismo match no se registre dos veces en distinto orden (A-B y B-A), los IDs siempre se normalizan antes de insertar: el ID menor va en `id_usuario1` y el mayor en `id_usuario2`. Esto se gestiona tanto en el DAO de Java mediante las funciones `Math.min()` y `Math.max()`, como en el trigger de la base de datos mediante las funciones `LEAST()` y `GREATEST()`.

```
CREATE TABLE Matches (  
    id_matches INT NOT NULL AUTO_INCREMENT,  
    id_usuario1 INT NOT NULL,  
    id_usuario2 INT NOT NULL,  
    fecha DATETIME NOT NULL DEFAULT NOW(),  
  
    PRIMARY KEY (id),  
    UNIQUE (id_usuario1, id_usuario2),  
  
    CONSTRAINT fk_match_u1 FOREIGN KEY (id_usuario1)  
        REFERENCES usuario(id_usuario) ON DELETE CASCADE,  
  
    CONSTRAINT fk_match_u2 FOREIGN KEY (id_usuario2)  
        REFERENCES usuario(id_usuario) ON DELETE CASCADE  
);
```

3.6. Trigger: after_like_insert

Su función es la siguiente. Cuando se inserta un nuevo like del usuario A hacia el usuario B, el trigger comprueba si ya existe en la tabla Likes el registro inverso (B → A). Si existe, significa que hay reciprocidad y, por lo tanto, se ha producido un match. En ese caso, el trigger verifica que el match no esté ya registrado en la tabla Matches para evitar duplicados, y si no existe, lo inserta normalizando los IDs con `LEAST()` y `GREATEST()`.

```
DELIMITER //
```

```
CREATE TRIGGER after_like_insert  
AFTER INSERT ON Likes  
FOR EACH ROW  
BEGIN  
    IF EXISTS (  
        SELECT 1 FROM Likes  
        WHERE id_emisor = NEW.id_receptor  
        AND id_receptor = NEW.id_emisor  
    ) THEN  
        IF NOT EXISTS (  
            SELECT 1 FROM Matches  
            WHERE (id_usuario1 = LEAST(NEW.id_emisor, NEW.id_receptor)  
                AND id_usuario2 = GREATEST(NEW.id_emisor, NEW.id_receptor))  
        ) THEN  
            INSERT INTO Matches (id_usuario1, id_usuario2)
```




```
VALUES (  
    LEAST(NEW.id_emisor, NEW.id_receptor),  
    GREATEST(NEW.id_emisor, NEW.id_receptor)  
);  
END IF;  
END IF;  
END //  
DELIMITER ;
```

3.7. Procedimientos Almacenados.

3.7.1. Insertar_usuarios

Insertar_usuarios sirve para insertar usuarios solo llamando al procedimiento almacenado pasándole los parámetros, nombre, correo, bio, password y tecnología.

```
DELIMITER //  
CREATE PROCEDURE insertar_usuario (  
IN p_nombre VARCHAR(50),  
IN p_correo VARCHAR(50),  
IN p_bio TEXT,  
IN p_password VARCHAR(50),  
IN Tecnologia VARCHAR(50)  
)  
BEGIN  
INSERT INTO usuario(nombre, correo, bio, password) VALUES (nombre,  
correo, bio, password);  
END  
DELIMITER ;
```

3.7.2. BorrarUsuario

Si llamamos a este procedimiento y le pasamos el id borra el usuario.

```
DELIMITER  
CREATE PROCEDURE BorrarUsuario(IN p_id_usuario INT)  
BEGIN  
  
    IF NOT EXISTS (SELECT 1 FROM usuario WHERE id_usuario = p_id_usuario)  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'El usuario no existe';  
    END IF;  
  
    DELETE FROM usuario  
    WHERE id_usuario = p_id_usuario;
```

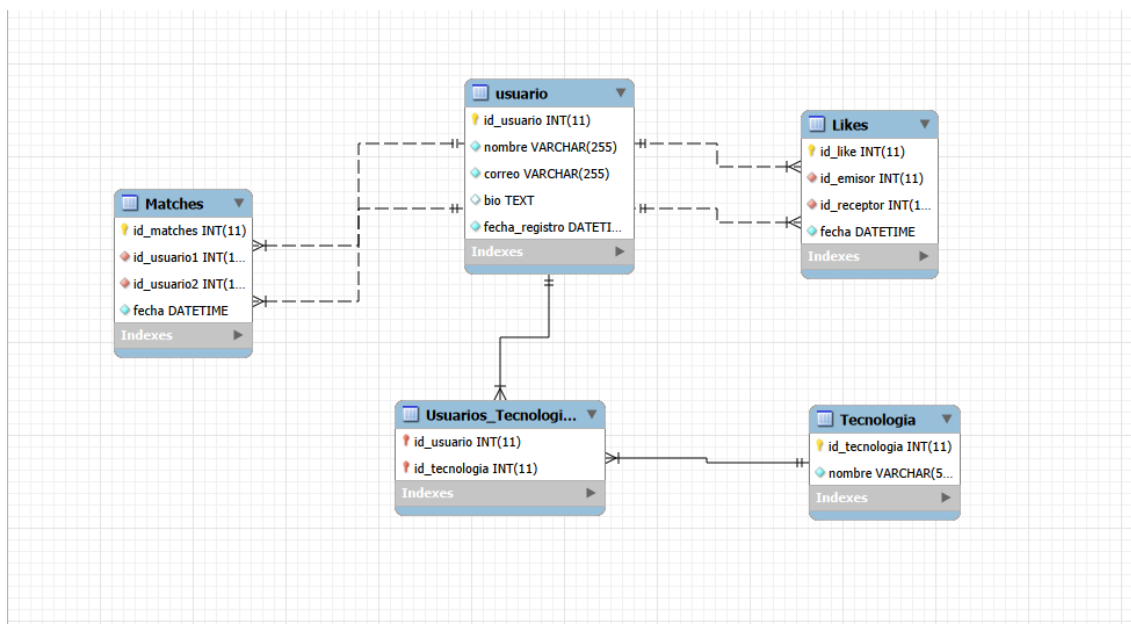


```
SELECT CONCAT('Usuario ', p_id_usuario, ' eliminado correctamente')
AS mensaje;
END
DELIMITER ;
```

3.7.3. CargarUsuario

```
4. DELIMITER
5. CREATE PROCEDURE CargarUsuario(
6.     IN p_nombre      VARCHAR(255),
7.     IN p_correo       VARCHAR(255),
8.     IN p_bio          TEXT
9. )
10. BEGIN
11.     IF p_nombre = '' OR p_nombre IS NULL THEN
12.         SIGNAL SQLSTATE '45000'
13.         SET MESSAGE_TEXT = 'El nombre no puede estar vacío';
14.     END IF;
15.
16.     IF p_correo = '' OR p_correo IS NULL THEN
17.         SIGNAL SQLSTATE '45000'
18.         SET MESSAGE_TEXT = 'El correo no puede estar vacío';
19.     END IF;
20.
21.     -- Verificar que el correo no esté ya registrado
22.     IF EXISTS (SELECT 1 FROM usuario WHERE correo = p_correo) THEN
23.         SIGNAL SQLSTATE '45000'
24.         SET MESSAGE_TEXT = 'El correo ya está registrado';
25.     END IF;
26.
27.     -- Insertar el nuevo usuario
28.     INSERT INTO usuario (nombre, correo, bio)
29.     VALUES (p_nombre, p_correo, p_bio);
30.
31.     -- Retornar el usuario creado
32.     SELECT
33.         id_usuario,
34.         nombre,
35.         correo,
36.         bio,
37.         fecha_registro
38.     FROM usuario
39.     WHERE id_usuario = LAST_INSERT_ID();
40. END
41. DELIMITER ;
```

3.8. Esquema Entidad Relación (EER)





4. Desarrollo del Backend (Java/Spring Boot)

Todo el código del servidor reside en el paquete `com.example.BackEnd`. La organización sigue el patrón de arquitectura en capas: por cada entidad del dominio existe una clase de modelo, una clase DAO y una clase Controller. Además, hay clases transversales de configuración y utilidades.

Clase	Tipo	Responsabilidad
Main.java	Arranque	Punto de entrada de Spring Boot. Lanza el servidor en el puerto 8080
CorsConfig.java	Configuración	Habilita CORS para todos los endpoints de <code>/api/**</code> , permitiendo peticiones desde el frontend
Variables.java	Utilidad	Centraliza la URL de conexión JDBC, usuario y contraseña de la base de datos
Usuario/UsuarioDAO/UsuarioController	Capa Completa	Gestión de registro, login, listado y consulta de perfiles de usuario.
Like/LikeDAO/LikeController	Capa Completa	Registro de likes y detección de match mutuo.
Mach/MachDAO/MachController	Capa Completa	Creación y consulta de matches con información del otro perfil.
Tecnologia/TecnologiaDAO/TecnologiaController	Capa Completa	Gestión del catálogo de tecnologías y vinculación usuario-tecnología

Tabla 2. Estructura de Java

4.1. Conexión JDBC

La conexión a la base de datos se realiza mediante JDBC (Java Database Connectivity), la API estándar de Java para la comunicación con bases de datos relacionales. Los parámetros de conexión se centralizan en la clase `Variables.java`, que expone tres campos estáticos públicos: `url` (cadena de conexión JDBC), `user` y `password`. Todos los DAOs instancian una conexión en cada método mediante `DriverManager.getConnection()`, dentro de un bloque `try-with-resources` que garantiza el cierre automático de la conexión al terminar la operación.

```
public class Variables {  
    public static String url = "jdbc:mysql://localhost:3306/LoveCode";  
    public static String user = "root";  
    public static String password = "root";  
}
```



4.2. Registro de Usuario

El proceso de registro está implementado `UsuarioController.registrar()` y `UsuarioDAO.insertarUsuario()`. Cuando el frontend envía un `POST /api/registro` con los datos del formulario en formato JSON, el controlador extrae los campos del cuerpo de la petición, valida que ninguno sea nulo, y delega en el DAO.

El DAO comprueba primero si ya existe un usuario con el mismo nombre o correo. Si existe alguno, devuelve `false` y el controlador responde con un código HTTP 409 Conflict. Si no hay duplicados, inserta el registro en la tabla `usuario` y luego procesa el campo `tecnologias`, que llega como cadena separada por comas (ej. "Java, React, SQL"). Para cada tecnología, el método `insertarTecnologiaUsuario()` implementa el patrón `get-or-insert`: busca la tecnología en el catálogo y, si no existe, la inserta; finalmente crea el vínculo en `Usuarios_Tecnologias` usando `INSERT IGNORE` para evitar duplicados.

4.3. Validación del Login

El endpoint `POST /api/login` recibe el correo y la contraseña del usuario. `UsuarioDAO.login()` realiza una consulta `SELECT` a la tabla `usuario` filtrando por ambos campos. Si el resultado es nulo (no existe ningún usuario con esa combinación correo-contraseña), el controlador responde con 401 Unauthorized. Si la consulta devuelve un registro, el controlador serializa en JSON el `id`, `nombre` y `correo` del usuario y los devuelve al frontend, que los persiste en `localStorage` para gestionar la sesión.

4.4. Obtención de Perfiles para el Dashboard

El endpoint `GET /api/usuarios` es consumido por el dashboard para cargar la pila de tarjetas. `UsuarioDAO.obtenerTodosLosUsuarios()` ejecuta una consulta que hace `JOIN` entre las tablas `usuario`, `Usuarios_Tecnologias` y `Tecnologia`, agrupando las tecnologías de cada usuario en una cadena separada por comas mediante `GROUP_CONCAT`. Esto permite devolver en un único objeto JSON por usuario todos sus datos incluyendo las tecnologías, sin necesidad de múltiples peticiones.

4.5. Registro del Likes y Detección de Match

El endpoint POST /api/likes es el núcleo funcional de la aplicación. Recibe un objeto JSON con los campos idOrigen (el usuario que da el like) e idDestino (el usuario que lo recibe). LikeController realiza dos validaciones previas: que ambos campos estén presentes y que idOrigen sea distinto de idDestino (no se puede dar like a uno mismo).

```
public boolean darLike(int origen, int destino) throws SQLException {
    if (yaExisteLike(origen, destino)) return false;

    String sql = "INSERT INTO Likes (id_emisor, id_receptor) VALUES
(?, ?)";
    try (Connection con = DriverManager.getConnection(Variables.url,
Variables.user, Variables.password);
        PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
        ps.setInt(1, origen);
        ps.setInt(2, destino);
        ps.executeUpdate();

        if (yaExisteLike(destino, origen)) {
            MachDAO machDAO = new MachDAO();
            machDAO.crearMatch(origen, destino);
        }
        return true;
    }
}
```

LikeDAO.darLike() ejecuta el proceso completo:

1. Comprueba con yaExisteLike(origen, destino) si ya existe ese like. Si existe, devuelve false sin insertar.
2. Inserta el registro en la tabla Likes.
3. Comprueba con yaExisteLike(destino, origen) si existe el like recíproco.
4. Si existe reciprocidad, llama a MachDAO.crearMatch() para registrar el match.
5. MachDAO.crearMatch() normaliza los IDs con Math.min/Math.max, verifica que el match no exista ya, y lo inserta.

El controlador devuelve al frontend un JSON con dos campos: insertado (boolean que indica si el like se registró) y match (boolean que indica si se ha producido una coincidencia mutua). El frontend usa este campo match para mostrar el toast de celebración "💖 ¡Es un match!".



4.6. Spring Boot

En el contexto de LoveCode, Spring Boot actúa como el servidor HTTP que recibe las peticiones del navegador, ejecuta la lógica de negocio (registro de usuarios, detección de matches, consulta de perfiles) y devuelve las respuestas en formato JSON. Sin Spring Boot, implementar todo esto en Java puro requeriría gestionar manualmente los sockets de red, parsear las cabeceras HTTP, serializar objetos a JSON y mucho más. Spring Boot abstrae toda esa complejidad y nos permite centrarnos en la lógica de la aplicación. Elegimos Spring Boot por tres razones principales: es el framework Java más usado en la industria (lo que lo convierte en una habilidad muy valorada en el mercado laboral), su curva de aprendizaje es razonable para el nivel del ciclo formativo, y su integración con Maven facilita la gestión de dependencias como el conector de MySQL y el servidor HTTP embebido Tomcat.

4.6.1 Pom.xml

El archivo pom.xml es el descriptor del proyecto Maven. Define las dependencias que Spring Boot necesita para funcionar.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
  </dependency>
</dependencies>
```

La dependencia spring-boot-starter-web incluye automáticamente el servidor Tomcat embebido, la librería Jackson para serializar objetos Java a JSON y todo el soporte para crear controladores REST. Esto significa que no necesitamos instalar ni configurar ningún servidor externo: al ejecutar la aplicación, Spring Boot arranca su propio servidor HTTP en el puerto 8080.



Una vez tengamos el spring boot en funcionamiento nos tendría que salir algo a si en la terminal:

```

Iniciando mi aplicación LoveCode con Spring Boot...

:: Spring Boot ::
(3.2.4)

2026-05-29T09:43:59.351402:00 INFO 14556 --- [main] com.example.BackEnd.Main : Starting Main using Java 26.0.1 with PID 14556 (C:\Users\PC00100\Documents\Java\codecrush\target\classes started by PC00100 in C:\Users\PC00100\Documents\Java\codecrush)
2026-05-29T09:43:59.356402:00 INFO 14556 --- [main] com.example.BackEnd.Main : No active profile set, falling back to 1 default profile: "default"
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/C:/Users/PC00100/.m2/repository/org/apache/tomcat/embed/tomcat-embed-core/10.1.19/tomcat-embed-core-10.1.19.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

2026-05-29T09:44:00.627402:00 INFO 14556 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-29T09:44:00.658402:00 INFO 14556 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-29T09:44:00.659402:00 INFO 14556 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.19]
2026-05-29T09:44:00.792402:00 INFO 14556 --- [main] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-05-29T09:44:00.794402:00 INFO 14556 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1337 ms
2026-05-29T09:44:01.268402:00 INFO 14556 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-05-29T09:44:01.279402:00 INFO 14556 --- [main] com.example.BackEnd.Main : Started Main in 2.53 seconds (process running for 3.001)

```

4.6.2 Punto de entrada/ Main.java

Toda aplicación Spring Boot tiene una clase principal anotada con `@SpringBootApplication` que actúa como punto de arranque.

```

package com.example.BackEnd;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class Main {
    public static void main(String[] args) {
        System.out.println("Iniciando mi aplicación LoveCode con Spring Boot...\n");
        SpringApplication.run(Main.class, args);
    }
}

```

- `@SpringBootApplication`: marca la clase como fuente de configuración de Spring.
- `@EnableAutoConfiguration`: activa la configuración automática. Spring Boot analiza las dependencias del pom.xml y configura automáticamente los componentes necesarios (servidor Tomcat, serialización JSON con Jackson, etc.).
- `@ComponentScan`: indica a Spring que escanee el paquete actual y sus subpaquetes en busca de clases anotadas con `@RestController`, `@Service`, `@Repository` u otras anotaciones de Spring.

Al ejecutar `SpringApplication.run()`, Spring Boot inicializa el contexto de la aplicación, descubre todos los controladores definidos en el proyecto, los registra como manejadores de rutas HTTP y arranca el servidor Tomcat embebido.



4.6.3 Configuración CORS

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad de los navegadores que bloquea las peticiones HTTP a un dominio diferente al de la página actual. Como el frontend de LoveCode se sirve desde un origen distinto al del backend (por ejemplo, desde Live Server en el puerto 5500 mientras el backend corre en el 8080), el navegador bloquearía todas las peticiones `fetch()` sin la configuración adecuada.

```
package com.example.BackEnd;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("*")
                    .allowedMethods("GET", "POST", "PUT", "DELETE",
"OPTIONS")
                    .allowedHeaders("*");
            }
        };
    }
}
```

La anotación `@Configuration` le indica a Spring que esta clase contiene definiciones de beans (componentes gestionados por Spring). La anotación `@Bean` sobre el método `corsConfigurer()` le indica que el objeto devuelto debe registrarse en el contexto de la aplicación y aplicarse globalmente.



5. Desarrollo del Frontend

El frontend está compuesto por cinco páginas HTML independientes, cada una con un propósito concreto dentro del flujo de la aplicación. Todas comparten la hoja de estilo global style.css, que define las variables CSS del sistema de diseño, los estilos de formularios, botones, animaciones de fondo y componentes reutilizables.

Página	JS asociado	Función
Index.html	–	Landing page con el logo, eslogan y accesos directos a login y registro
Login.html	Formlogin.js	Formulario de autenticación con campos de correo y contraseña
Registrar.html	Formregistrar.js	Formulario de registro con chips de selección de tecnologías
Dashboard.html	Dashboardfuncion.js	Pantalla principal con el motor de swipe y botones de acción
Maches.html	Machesfuncion.js	Grid de matches con información completa del otro perfil

Tabla 3. Estructura del frontend

El formulario de registro utiliza checkboxes HTML ocultos con labels visibles que actúan como chips seleccionables. Mediante CSS, cuando un checkbox está marcado, su label asociado recibe un gradiente de fondo morado-rosa que lo resalta visualmente. Formregistrar.js mapea los valores abreviados de los checkboxes (ej. "js", "py") a los nombres completos de las tecnologías ("JavaScript", "Python") antes de enviarlos al backend.



5.1. Sistema de Diseño Visual

La identidad visual de LoveCode se articula en torno a una paleta de colores oscura con tonos morados y rosas, definida mediante variables CSS en el selector `:root` de `style.css`. Esto garantiza la coherencia visual entre todas las páginas y facilita cambios globales de tema con una sola modificación.

```
:root {
  --purple-dark:    #1a0a2e;
  --purple-accent:  #7c3aed;
  --pink:           #e040fb;
  --pink-soft:      #f472b6;
  --white:          #f5f0ff;
  --gray:           #c4b5d8;
}
```

5.2. Peticiones HTTP Asincronas con `fetch()`

Toda la comunicación con el backend se realiza mediante la función nativa `fetch()` del navegador, disponible en todos los navegadores modernos sin necesidad de librerías externas. Cada módulo JavaScript define la constante `API_BASE = "http://localhost:8080/api"` y la usa como prefijo para todas las peticiones.

Las peticiones POST incluyen la cabecera `Content-Type: application/json` y el cuerpo serializado con `JSON.stringify()`. Las respuestas se parsean con `response.json()` y se usan para actualizar el DOM o redirigir al usuario. Todos los bloques `fetch` están envueltos en `try-catch` para gestionar errores de red y mostrar mensajes informativos al usuario.

```
fetch(BASE_URL + '/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ correo, password }) // ← clave "correo"
})
.then(r => r.json())
.then(data => {
  if (data.error) {
    alert('✗ ' + data.error);
  } else {
    localStorage.setItem('userId', data.id);
    localStorage.setItem('userNombre', data.nombre);
    localStorage.setItem('userCorreo', data.correo);
    alert('☑ Bienvenido, ' + data.nombre);
    window.location.href = 'Dashboard.html';
  }
})
})
```

5.3. Motor de Swipe

El motor de swipe es el componente técnicamente más complejo del frontend. Está implementado íntegramente en JavaScript vanilla, sin ninguna librería externa, y gestiona tanto eventos de ratón como eventos táctiles para funcionar correctamente en dispositivos móviles y de escritorio. Al cargarse el dashboard, `cargarPerfiles()` obtiene todos los usuarios de la API, filtra el perfil del propio usuario autenticado comparando los IDs (usando `parseInt` para evitar problemas de comparación entre strings y números), y construye el stack visual con `renderStack()`. El stack muestra hasta tres tarjetas apiladas con diferentes niveles de z-index, opacidad y transformación CSS para simular profundidad.

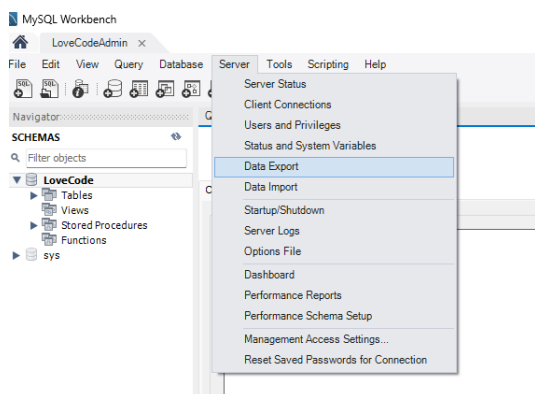
El gesto de arrastre se gestiona con tres eventos: `mousedown/touchstart` registra la posición inicial; `mousemove/touchmove` aplica la transformación CSS en tiempo real (`translate + rotate` proporcional al desplazamiento); `mouseup/touchend` evalúa el desplazamiento final y decide la acción. Si el desplazamiento horizontal supera los 100 píxeles hacia la derecha, se ejecuta un like; si supera los 100 píxeles hacia la izquierda, se pasa el perfil. Durante el arrastre, los stamps LIKE y NOPE aparecen con opacidad creciente a partir de los 30 píxeles de desplazamiento, proporcionando retroalimentación visual inmediata. Adicionalmente, las teclas de flecha del teclado (`ArrowRight`, `ArrowLeft`, `ArrowUp`) desencadenan las mismas acciones que el swipe, mejorando la accesibilidad en entornos de escritorio.

6. Backup

Se han realizado tres tipos de backups diferentes:

6.1. Backup desde entorno grafico

Paso 1:



Para hacer el backup desde el entorno grafico en este caso MySQL Workbench.



Paso dos:

Query 1 Administration - Data Export x

LoveCodeAdmin
Data Export

Advanced Options...

Object Selection Export Progress

Tables to Export

Exp... Schema
LoveCode
sys

Exp... Schema Objects

Refresh

Dump Structure and Dat Select Views Select Tables Unselect All

Objects to Export

☐ Dump Stored Procedures and Functions ☐ Dump Events ☐ Dump Triggers

Export Options

☒ Export to Dump Project Folder C:\Users\PC00100\Documents\dumps\Dump20260529 ...

Each table will be exported into a separate file. This allows a selective restore, but may be slower.

☐ Export to Self-Contained File C:\Users\PC00100\Documents\dumps\Dump20260529.sql ...

All selected database objects will be exported into a single, self-contained file.

☐ Create Dump in a Single Transaction (self-contained file only) ☐ Include Create Schema

Press [Start Export] to start...

Start Export

Seleccionamos la base de datos a la que le queremos hacer el backup y le daremos a start export.

Query 1 Administration - Data Export x

LoveCodeAdmin
Data Export

Advanced Options...

Object Selection Export Progress

Export Completed

Status:
5 of 5 exported.

Log:

```
11:30:03 Dumping LoveCode (Likes)
Running: mysqldump.exe --defaults-file="C:\Users\PC00100\AppData\Local\Temp\lmpkvq2ab3.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "LoveCode" "Likes"
11:30:03 Dumping LoveCode (Usuario)
Running: mysqldump.exe --defaults-file="C:\Users\PC00100\AppData\Local\Temp\lmpkvq2ab3.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "LoveCode" "Usuario"
11:30:03 Dumping LoveCode (Tecnologia)
Running: mysqldump.exe --defaults-file="C:\Users\PC00100\AppData\Local\Temp\lmpkvq2ab3.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "LoveCode" "Tecnologia"
11:30:03 Dumping LoveCode (Matches)
Running: mysqldump.exe --defaults-file="C:\Users\PC00100\AppData\Local\Temp\lmpkvq2ab3.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "LoveCode" "Matches"
11:30:03 Dumping LoveCode (Usuarios_Tecnologias)
Running: mysqldump.exe --defaults-file="C:\Users\PC00100\AppData\Local\Temp\lmpkvq2ab3.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "LoveCode" "Usuarios_Tecnologias"
11:30:04 Export of C:\Users\PC00100\Documents\dumps\Dump20260529 has finished
```

Stop Export Again

Una vez demos a start export nos saldrá esta pantalla, si no salta ningún error tendremos que ir a la carpeta de dumps y tendremos el backup hecho.



6.2. Backup manual desde la powershell

Una vez tenemos la maquina virtual con la configuración de red hecha correctamente. Abrimos una powershell y creamos una carpeta donde se va a guardar el backup. Una vez tengamos la carpeta creada ponemos este comando,

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" -h [La ip de la maquina] -u backup_user -p [nombre de la base de datos] > C:\Backups\backup_nombre_dela basededatos.sql
```

6.3. Backup Automatizado

Tendremos que crear un archivo llamado backup_base_datos.ps1.

```
# CONFIGURACION
$mysqlDump = "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe"
$servidor = "192.168.16.3"
$usuario = "backup_user"
$password = "Backup1234!"
$baseDatos = "love_code"
$carpetaBackup = ""

# CREAR CARPETA SI NO EXISTE
if (!(Test-Path -Path $carpetaBackup)) {
    New-Item -ItemType Directory -Path $carpetaBackup
}

# fecha para el nombre del archivo
$fecha = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$archivoBackup = "$carpetaBackup\backup_${LoveCode}_$fecha.sql"

# HACER BACKUP
&$mysqlDump --host=$servidor --user=$usuario --password=$password $baseDatos > $archivoBackup

# COMPROBAMOS SI SE HA CREADO EL BACKUP
if (Test-Path -Path $archivoBackup) {
    Write-Host "Backup creado exitosamente: $archivoBackup"
} else {
    Write-Host "Error al crear el backup."
}
```

Después para ejecutarlos tendremos que poner este comando cd

```
C:\.\backup_base_datos.ps1
```

Después tendremos que irnos a el programador de tareas, tendremos que crear una tarea básica y le pondremos de nombre Backup automático base de datos.



También tendremos que elegir la frecuencia con la que se va a hacer el backup por ejemplo Diaria. En acción pondremos Iniciar un programa y en programa pondremos powershell.exe.

7. Docker

Debido a varios problemas que se han detectado en la virtual box se ha tenido que buscar una alternativa. Se han montado dos contenedores de Docker uno donde se aloja la base de datos y otro donde se aloja el servidor nginx.

Para levantar el contenedor de la base de datos lo primero que tenemos que hacer es un Docker-compose.yaml

```
services:
  mysql:
    image: mysql:8.0
    container_name: LoveCodeMySQL
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: LoveCode
      MYSQL_USER: admin
      MYSQL_PASSWORD: admin
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

☐		dockerdatabase	-	-	-	1.16%	1 hour ago				
☐		LoveCodeMySQL	0fe92e951b40	mysql:8.0	3306:3306	1.16%	1 hour ago				

Donde ponemos ver que esta tanto la imagen es la de MySQL como el nombre que queremos que tenga el contenedor, como la configuración del usuario para poder editar la base de datos, El puerto por defecto que es el 3306.

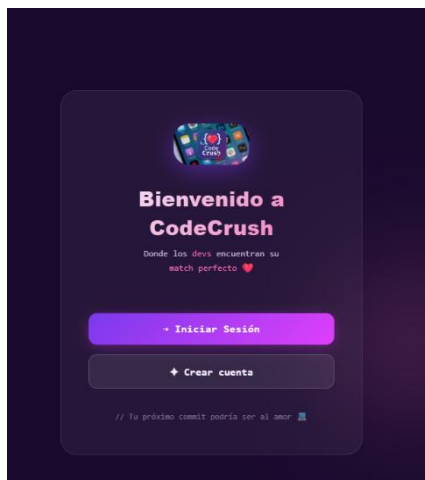
Para encender el contenedor tendremos que ejecutar el comando Docker-compose up -d

```
docker run --name mi-servidor-nginx -p 5500:80 -v
/ruta/de/tu/carpeta:/usr/share/nginx/html:ro -d nginx
```

Container ID	Image	Port(s)	CPU (%)	Last started	Actions
3625d37246cc	nginx	5500:80	0%	1 day ago	

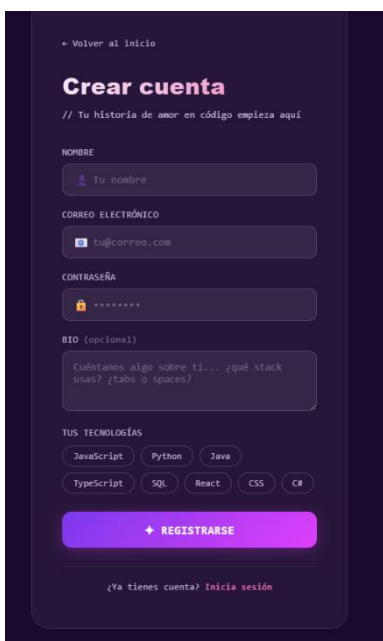
8. Funcionamiento de la web

Paso 1 | Acceso a la aplicación



El usuario abre el navegador y accede a index.html. Encuentra la landing page de LoveCode con el logo, el eslogan "Donde los devs encuentran su match perfecto" y dos botones: Iniciar Sesión y Crear cuenta.

Paso 2 | Registro



El usuario hace clic en Crear cuenta y accede a Registrer.html. Rellena el formulario con su nombre, correo electrónico, contraseña y una breve bio. A continuación selecciona sus tecnologías haciendo clic en los chips interactivos (cada clic activa o desactiva visualmente el chip con un gradiente de color). Al pulsar el botón REGISTRARSE, Formregistrer.js valida que al menos una tecnología esté seleccionada, mapea los valores de los checkboxes a nombres completos y envía la petición POST /api/registro al backend. Si el registro es exitoso, el usuario es redirigido automáticamente a login.html.

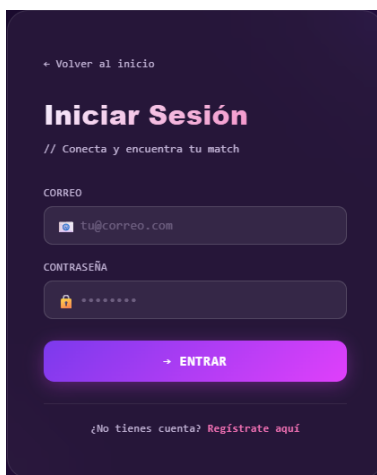
Paso 4 | Exploración de perfiles



El dashboard carga los perfiles de otros desarrolladores, excluyendo automáticamente el perfil del propio usuario autenticado. Los perfiles se presentan en forma de tarjetas apiladas que muestran el nombre, el handle (parte local del correo), la bio y los chips de tecnologías. El usuario puede interactuar de tres formas: arrastrar la tarjeta hacia la derecha para dar like, hacia la izquierda para pasar, o hacia arriba para saltar. También puede usar los botones ♥, ✕ y ↻, o las teclas de flecha del teclado.



Paso 3 | Inicio de Sesión



← Volver al inicio

Iniciar Sesión

// Conecta y encuentra tu match

CORREO

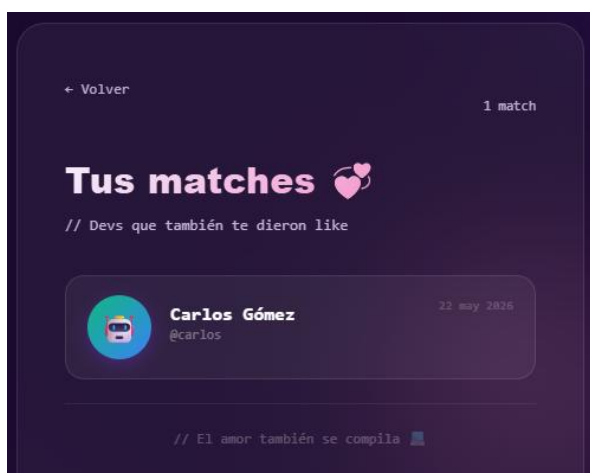
CONTRASEÑA

→ ENTRAR

¿No tienes cuenta? [Regístrate aquí](#)

El usuario introduce su correo y contraseña en login.html. Formlogin.js envía la petición POST /api/login. Si las credenciales son correctas, el backend responde con el id, nombre y correo del usuario, que se almacenan en localStorage. El usuario es redirigido automáticamente a Dashboard.html.

Paso 5 | Like y detención de Match



← Volver 1 match

Tus matches

// Devs que también te dieron like



Carlos Gómez
@carlos

22 may 2026

// El amor también se compila

Al deslizar una tarjeta hacia la derecha, el frontend envía inmediatamente una petición POST /api/likes con el ID del usuario autenticado y el ID del perfil que acaba de recibir el like. El backend registra el like y comprueba si existe el like recíproco. Si es así, crea el match y devuelve match: true en la respuesta. El frontend detecta este campo y muestra el toast de celebración "💖 ¡Es un match!" con un retraso de 500 ms para que coincida con el final de la animación de la tarjeta.



9. Problemas Y soluciones.

Fallo en maquina virtual

Uno de los problemas fue al configurar la maquina virtual. La tarjeta de red se configuro en Solo Maquina – Anfitrión. No se comunicaba con la maquina anfitrión y la solución fue cargar la base de datos en un docker.

Fallo en tabla usuario

Otro fallo Fue en la base de datos se creo la tabla usuario y no se creó la columna de password, entonces cuando el java iba a insertar la contraseña fallaba. La solución fue hacer un alter table e implementar la tabla.

Error 500 en el endpoint de matches.

Al acceder a la página de matches, el navegador mostraba un error HTTP 500 Internal Server Error en la petición a GET /api/matches/{id}. Revisando los logs del servidor Spring Boot, el mensaje de error indicaba que la tabla 'lovecode.matches' no existía.

El problema era una inconsistencia entre mayúsculas y minúsculas en el nombre de la tabla. En la base de datos los nombres de tablas son case-sensitive por defecto. La tabla se había creado como 'Matches' (con M mayúscula), pero la query SQL en MachDAO.java usaba 'matches' en minúscula en la cláusula FROM. En Windows esto no da error porque el sistema de archivos es case-insensitive, pero en el servidor Linux sí fallaba.

La solución fue corregir la query en MachDAO.java para usar el nombre exacto de la tabla con la mayúscula correcta: FROM Matches m. Esto nos enseñó la importancia de ser consistentes con los nombres de objetos de base de datos y de probar siempre en un entorno Linux, que es el entorno habitual de producción.

Los Likes no se insertaban en la base de datos

Durante las pruebas de integración, al dar like a un perfil el toast de "Like enviado" aparecía correctamente en la interfaz, pero al revisar la tabla Likes en la base de datos el registro nunca aparecía. No había ningún error en la consola del navegador ni en los logs del backend.

La investigación reveló que el problema estaba en la extracción del ID del perfil en el JavaScript del dashboard. La línea que obtenía el ID usaba perfil?.id_usuario ?? perfil?.id, pero Spring Boot serializaba el campo como id (en minúscula, derivado del getter getId() de la clase Java). Como id_usuario era undefined y id valía algo, la cadena de nullish coalescing devolvía el valor correcto... pero solo en algunos casos. El problema real era que si el ID era 0 (falsy en JavaScript), el operador ?? no se activaba correctamente con el operador &&.

La solución definitiva fue usar una comprobación explícita con != null en lugar de depender de la falsiness del valor: const idDestino = perfil?.id ?? perfil?.id_usuario ?? perfil?.idUsuario, con if (idDestino != null) antes de llamar a darLike(). Además se añadieron console.log en cada punto del flujo para diagnosticar el problema, lo que resultó una técnica de depuración muy eficaz para localizar exactamente en qué paso se cortaba la ejecución



10. Conclusion

LoveCode ha sido mucho más que un proyecto de evaluación. Ha sido nuestra primera aproximación real a lo que significa desarrollar software de principio a fin: analizar requisitos, diseñar una arquitectura, implementar, depurar, integrar y documentar. Las dificultades encontradas en el camino no han sido obstáculos sino oportunidades de aprendizaje, y el resultado final es un sistema funcional del que nos sentimos legítimamente orgullosos.

En cuanto a las mejoras futuras, identificamos varias líneas de desarrollo que harían de LoveCode una aplicación más robusta y completa:

- Seguridad: implementar hashing de contraseñas con BCrypt y autenticación basada en tokens JWT para proteger los endpoints de la API.
- Chat en tiempo real: tras producirse un match, habilitar una sala de chat entre los dos usuarios usando WebSockets con Spring WebSocket y Stomp, permitiendo la comunicación directa dentro de la plataforma.
- Filtrado inteligente: añadir un sistema de filtrado en el dashboard que permita ver primero los perfiles con más tecnologías en común, o filtrar por tecnología específica.
- Foto de perfil: permitir la subida de imágenes para personalizar los avatares, que actualmente se generan algorítmicamente.
- Paginación: implementar paginación en el endpoint `/api/usuarios` para escalar correctamente cuando la plataforma tenga miles de usuarios registrados.
- Pruebas automatizadas: añadir tests unitarios con JUnit 5 y Mockito para los DAOs y Controllers, e integrar un pipeline de integración continua con GitHub Actions.